

CCE 333 — Database Systems

Third Year — Computer and Control Engineering

DATABASE DESIGN REPORT

Freelance Marketplace System

Database Design and Implementation Report

Scope. Relational database analysis, EER-to-relational mapping, normalization to 3NF, SQL implementation, and query demonstration for a freelance marketplace platform.

Prepared by: Noureden Elhakiem
Project Type: Academic Database Project
Portfolio: [GitHub](#) | [LinkedIn](#)

Repository-ready academic report

Contents

1	Project Overview	4
2	Problem Statement and Benefits	5
2.1	Problem Statement	5
2.2	Benefits of the Database Solution	6
3	Database Design Diagrams	7
4	Normalization	11
4.1	Unnormalized Form (UNF)	11
4.1.1	Problems Identified	11
4.2	First Normal Form (1NF)	12
4.3	Second Normal Form (2NF)	13
4.3.1	Partial Dependencies Identified	13
4.3.2	Decomposition into 2NF	13
4.3.3	Why 2NF Is Satisfied	14
4.4	Third Normal Form (3NF)	15
4.4.1	Transitive Dependency Analysis	15
4.4.2	3NF Result	15
4.5	Normalization Summary	16
5	Final Normalized Schema (3NF)	17
5.1	Relationship Summary	17
5.2	Key Constraints	18
6	SQL Implementation	19
6.1	Database Creation	19
6.2	Table Definitions	19
6.2.1	User Table	19
6.2.2	Client Table	19
6.2.3	Freelancer Table	20
6.2.4	Project Table	20
6.2.5	Proposal Table	21
6.2.6	Contract Table	21
6.2.7	Skill Table	21
6.2.8	Freelancer_Skill Table (Junction Table)	22
6.3	Sample Data Insertion	23
6.3.1	Inserting Users	23
6.3.2	Inserting Clients and Freelancers	23
6.3.3	Inserting Projects	24
6.3.4	Inserting Proposals	24
6.3.5	Inserting Contracts	25
6.3.6	Inserting Skills and Assignments	25
6.4	SELECT Queries with JOINS	26
6.4.1	List All Clients with User Information	26
6.4.2	List All Freelancers with Bio	26

6.4.3	List All Projects with Client Names	26
6.4.4	List All Proposals with Freelancer and Project Information	27
6.4.5	List All Contracts with Full Details	27
6.4.6	List All Freelancer Skills (M:N Traversal)	28
6.4.7	Count Proposals per Project (Aggregation)	28
6.4.8	Find Freelancers with a Specific Skill	28
6.5	UPDATE Examples	29
6.5.1	Update a Project Status	29
6.5.2	Update a Proposal Status to Accepted	29
6.5.3	Update a Freelancer's Bio	29
6.5.4	Update Contract End Date and Status	29
6.5.5	Update a Freelancer's Skill Proficiency	29
6.6	DELETE Examples	30
6.6.1	Delete a Freelancer-Skill Assignment	30
6.6.2	Delete a Contract	30
6.6.3	Delete a Proposal (Respecting FK Order)	30
6.6.4	Delete a Skill (With Dependent Records)	30
7	Conclusion	31
7.1	Database Design	31
7.2	Relational Schema	31
7.3	Normalization	31
7.4	SQL Implementation	31
7.5	Data Integrity	31

1 Project Overview

The **Freelance Marketplace System** is a relational database designed to model and manage the core operations of an online freelancing platform. The database captures the complete lifecycle of freelance engagements, from user registration through project completion.

The system revolves around the following key entities and their interactions:

- **Users** — The base entity representing all registered individuals in the system. Each user is classified as either a Client or a Freelancer, implementing an ISA (specialization) hierarchy.
- **Clients** — A specialization of User. Clients are the individuals or organizations that post projects and seek freelance services.
- **Freelancers** — A specialization of User. Freelancers are professionals who browse available projects and submit proposals offering their services. Each freelancer maintains a biographical profile.
- **Projects** — Work listings created by clients. Each project includes a title, description, budget, posting date, and current status (Open, In Review, or Closed). A project belongs to exactly one client, while a client may post multiple projects.
- **Proposals** — Bids submitted by freelancers on specific projects. Each proposal includes a cover letter, bid amount, submission date, and status (Pending, Accepted, or Rejected). A proposal links one freelancer to one project.
- **Contracts** — Formal agreements created from accepted proposals. Each contract records start and end dates along with a status. The one-to-one relationship between Contract and Proposal is enforced through a UNIQUE constraint on the foreign key.
- **Skills** — A catalog of professional competencies (e.g., PHP, MySQL, Graphic Design, Content Writing).
- **Freelancer Skills** — A many-to-many junction table that associates freelancers with their skills, including a proficiency level attribute (Beginner, Intermediate, or Advanced) for each assignment.

The database comprises **8 tables** implementing various relationship types: one-to-one (User–Client, User–Freelancer, Proposal–Contract), one-to-many (Client–Project, Freelancer–Proposal, Project–Proposal), and many-to-many (Freelancer–Skill through the junction table). This design provides a comprehensive demonstration of fundamental relational database concepts.

2 Problem Statement and Benefits

2.1 Problem Statement

Managing a freelance marketplace without a structured database leads to significant data management challenges. When client information, project listings, freelancer profiles, proposals, contracts, and skill records are maintained in unstructured formats (such as spreadsheets or flat files), the following problems arise:

1. **Data Redundancy** — Client and freelancer information is duplicated across multiple records whenever they participate in different projects or proposals, wasting storage and creating inconsistency risks.
2. **Update Anomalies** — Changing a freelancer's email address or a client's name requires locating and modifying every occurrence of that data, which is error-prone and time-consuming.
3. **Insertion Anomalies** — Recording a new skill is impossible without simultaneously associating it with a freelancer, and registering a new client requires a project context even if no project exists yet.
4. **Deletion Anomalies** — Removing the only proposal for a project might inadvertently delete the project information itself, or deleting a freelancer's last skill assignment could lose the skill record entirely.
5. **Lack of Referential Integrity** — Without foreign key constraints, there is no mechanism to prevent orphaned records such as proposals referencing non-existent projects or contracts linked to deleted proposals.
6. **Difficulty in Querying** — Extracting meaningful information, such as finding all freelancers with a specific skill or listing all proposals for a given project, becomes complex and unreliable without proper relational structure.

2.2 Benefits of the Database Solution

The Freelance Marketplace database addresses these problems through proper relational design:

1. **Elimination of Redundancy** — Normalization to Third Normal Form ensures that each piece of information is stored exactly once. User data resides only in the User table, project data only in the Project table, and so forth.
2. **Data Integrity through Constraints** — Primary keys guarantee entity uniqueness, foreign keys enforce valid references between related tables, and the UNIQUE constraint on Contract.proposal_id ensures the one-to-one relationship between contracts and proposals.
3. **Clean Entity Separation** — The ISA hierarchy (User $\rightarrow$ Client/Freelancer) allows shared attributes to be stored once in the User table while type-specific attributes (such as Freelancer.bio) are stored in the appropriate subtype table.
4. **Flexible Skill Management** — The many-to-many relationship through Freelancer_Skill with its proficiency_level attribute allows freelancers to have multiple skills and skills to belong to multiple freelancers, each with an independent proficiency rating.
5. **Anomaly-Free Operations** — Proper decomposition ensures that insertions, updates, and deletions can be performed on individual entities without unintended side effects on other data.
6. **Efficient Querying** — The relational structure supports JOIN operations that efficiently combine data across tables, enabling complex queries such as finding all proposals for a client's projects or listing all skills of a specific freelancer.

3 Database Design Diagrams

This section presents the three core design artifacts of the Freelance Marketplace database. Each diagram is displayed on a dedicated landscape page for clarity and readability.

- **Relational Schema** — The mapping of all tables, columns, primary keys, foreign keys, and their relationships.
- **Entity Relationship Diagram (ERD)** — The conceptual model showing entities, attributes, and relationships.
- **Enhanced Entity Relationship Diagram (EERD)** — The extended model incorporating the ISA specialization hierarchy for User, Client, and Freelancer.

Relational Schema

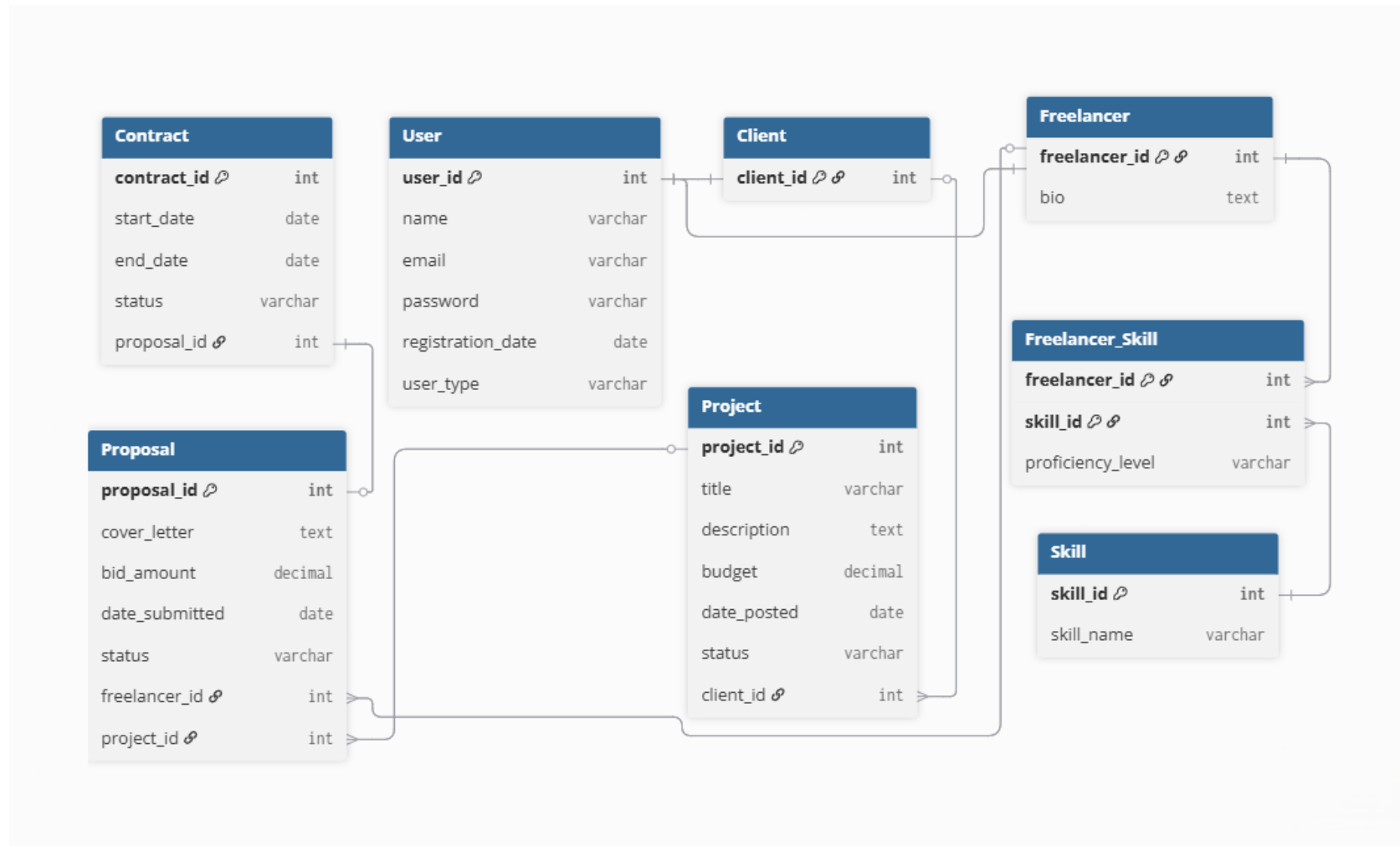


Figure 1: **Relational Schema** — Freelance Marketplace System

Entity Relationship Diagram (ERD)

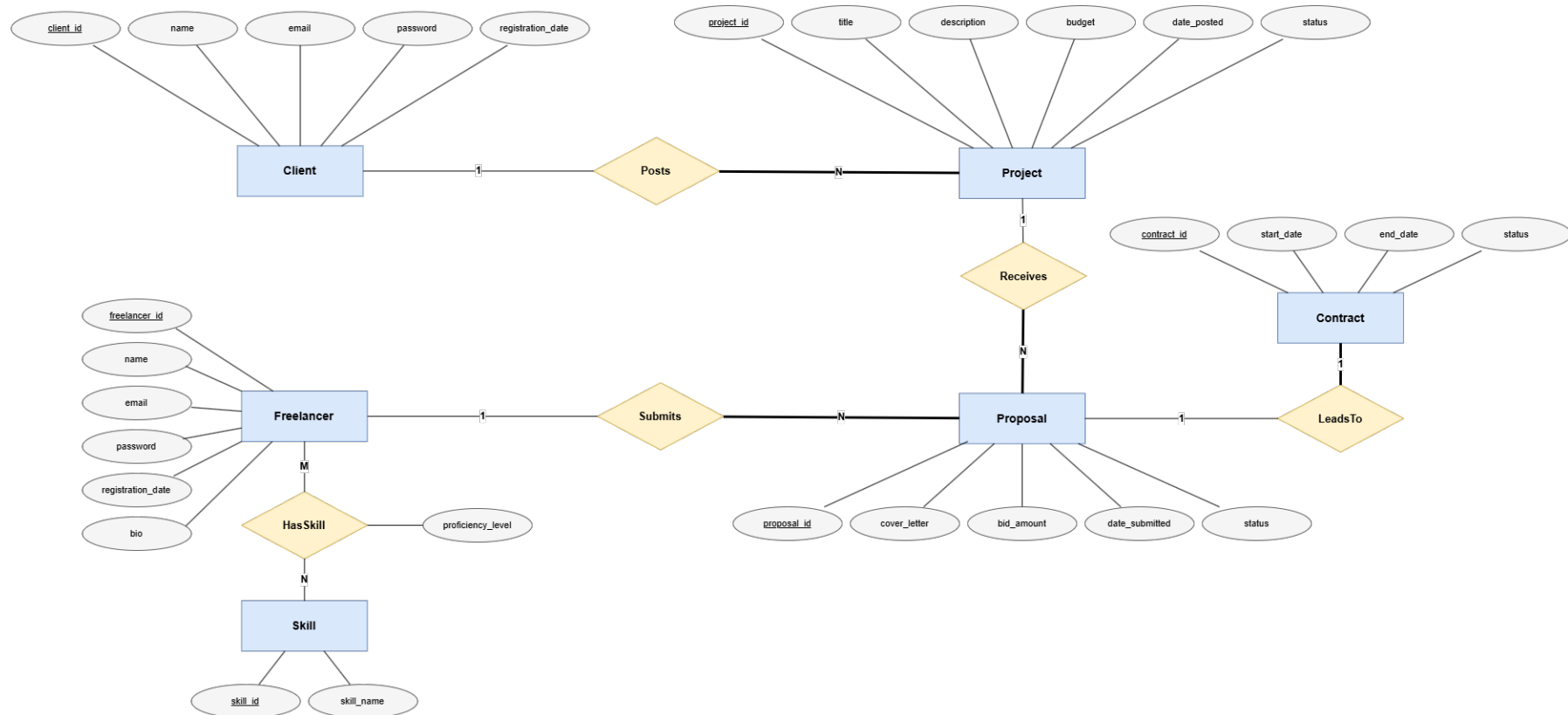


Figure 2: **Entity Relationship Diagram (ERD)** — Freelance Marketplace System

Enhanced Entity Relationship Diagram (EERD)

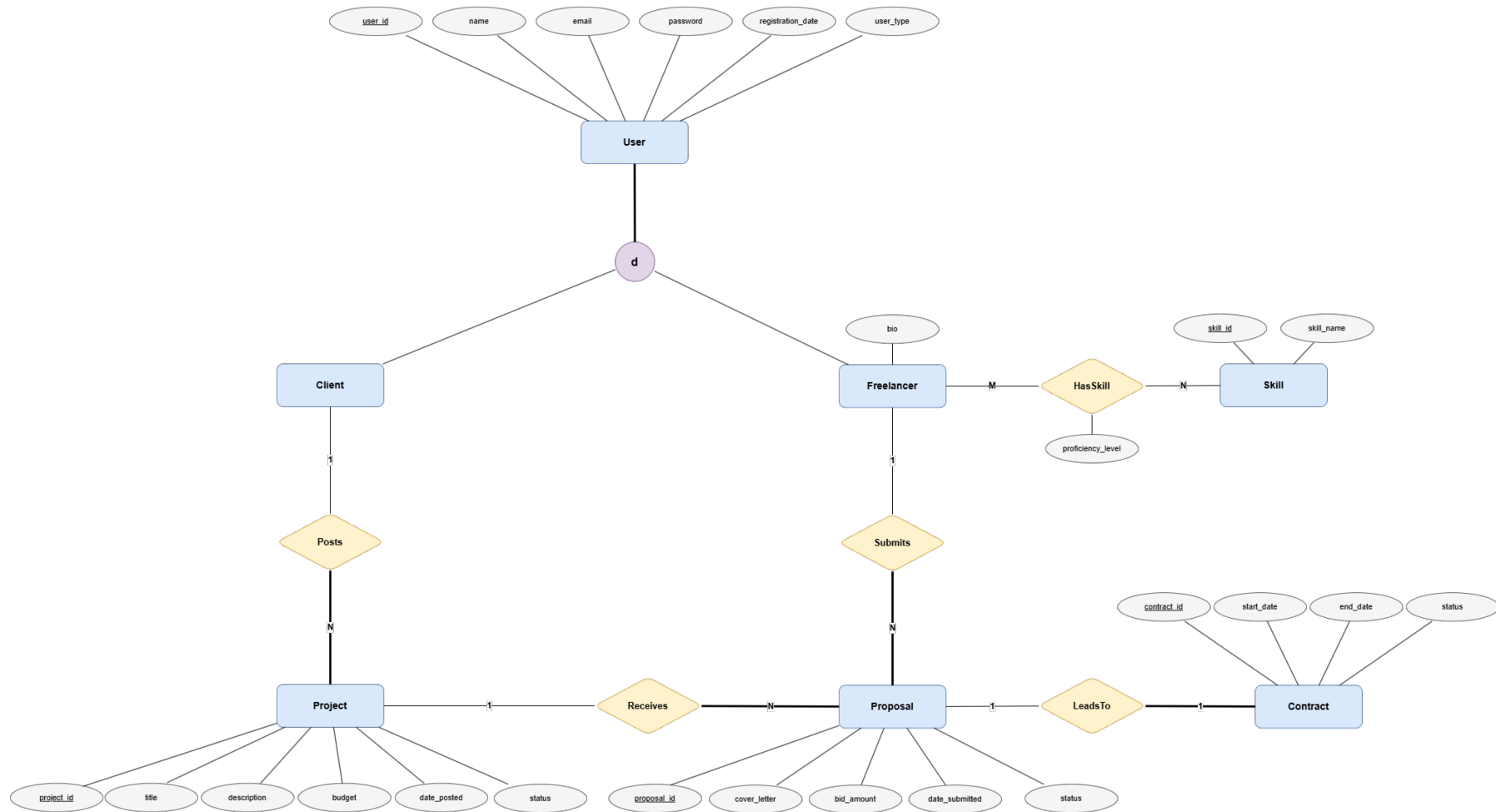


Figure 3: Enhanced Entity Relationship Diagram (EERD) — Freelance Marketplace System

4 Normalization

This section demonstrates the step-by-step normalization process applied to the Freelance Marketplace database, progressing from an Unnormalized Form (UNF) through First Normal Form (1NF), Second Normal Form (2NF), and finally Third Normal Form (3NF). The process validates that the final eight-table schema is free from redundancy and anomalies.

4.1 Unnormalized Form (UNF)

All system data is represented in a single flat table. Selected columns are shown for readability; the full UNF table also includes `email`, `password`, `registration_date`, `bio`, `description`, `budget`, `date_posted`, `project_status`, `cover_letter`, `bid_amount`, `date_submitted`, `proposal_status`, `contract_id`, `start_date`, `end_date`, and `contract_status`.

Table 1: UNF — Single Flat Table (selected columns)

<code>user_id</code>	<code>name</code>	<code>user_type</code>	<code>project_id</code>	<code>title</code>	<code>skills</code>	<code>prof_levels</code>
1	Ali	Client	10	Website	—	—
2	Sara	Freelancer	10	Website	{HTML,CSS,JS}	{Exp,Adv,Exp}
2	Sara	Freelancer	11	App	{HTML,CSS,JS}	{Exp,Adv,Exp}

4.1.1 Problems Identified

1. **Repeating groups:** `skills` and `prof_levels` hold multiple values in a single cell, violating the atomic value requirement.
2. **Redundancy:** User, project, and proposal data is duplicated across rows whenever a freelancer has multiple skills or participates in multiple projects.
3. **Update anomaly:** Changing Sara’s email requires updating every row she appears in, risking inconsistency if any row is missed.
4. **Insertion anomaly:** A new skill cannot be recorded without an associated proposal and project context.
5. **Deletion anomaly:** Deleting the only proposal for a project may inadvertently lose the project data entirely.

4.2 First Normal Form (1NF)

Action applied. The set-valued columns `skills` and `prof_levels` are eliminated by placing each individual skill on its own row. All attributes now contain only atomic (indivisible) values.

Table 2: 1NF — Repeating Groups Removed (selected columns)

<code>user_id</code>	<code>name</code>	<code>project_id</code>	<code>title</code>	<code>proposal_id</code>	<code>bid_amount</code>	<code>skill_name</code>	<code>prof_level</code>
1	Ali	10	Website	100	450	—	—
2	Sara	10	Website	100	450	HTML	Expert
2	Sara	10	Website	100	450	CSS	Advanced
2	Sara	10	Website	100	450	JS	Expert
2	Sara	11	App	101	700	HTML	Expert
2	Sara	11	App	101	700	CSS	Advanced
2	Sara	11	App	101	700	JS	Expert

Composite primary key: { `proposal_id`, `skill_name` }

1NF is satisfied because every cell now contains a single atomic value, and a composite primary key uniquely identifies each row. However, significant redundancy remains due to partial dependencies on the composite key.

4.3 Second Normal Form (2NF)

Rule. A relation is in 2NF if it is in 1NF **and** every non-key attribute is **fully functionally dependent** on the entire primary key (i.e., no partial dependencies exist).

4.3.1 Partial Dependencies Identified

Given the composite key { proposal_id, skill_name }:

Table 3: Functional dependencies causing partial dependency violations

FD	Functional Dependency	Type
FD1	<u>proposal_id</u> → cover_letter, bid_amount, date_submitted, proposal_status	Partial
FD2	<u>proposal_id</u> → freelancer_id, project_id	Partial
FD3	<u>project_id</u> → title, description, budget, date_posted, project_status	Partial
FD4	<u>user_id</u> → name, email, password, registration_date, user_type	Partial
FD5	<u>freelancer_id</u> → bio	Partial
FD6	<u>contract_id</u> → start_date, end_date, contract_status	Partial
FD7	{ <u>freelancer_id</u> , <u>skill_name</u> } → proficiency_level	Full

4.3.2 Decomposition into 2NF

Each partial dependency is removed into its own relation. The decomposition produces the following eight relations:

R1 — User

<u>user_id</u>	name	email	password	registration_date	user_type
1	Ali	ali@mail.com	p1	2024-01-10	Client
2	Sara	sara@mail.com	p2	2024-01-15	Freelancer

R2 — Client

<u>client_id</u> (FK → User.user_id)
1

R3 — Freelancer

<u>freelancer_id</u> (FK → User.user_id)	bio
2	Web dev

R4 — Project

<u>project_id</u>	title	description	budget	date_posted	status	client_id (FK)
10	Website	Build site	500	2024-02-01	Open	1
11	App	Build app	800	2024-03-01	Open	1

R5 — Proposal

<u>proposal_id</u>	cover_letter	bid_amount	date_submitted	status	freelancer_id (FK)	project_id (FK)
100	I can do it	450	2024-02-05	Accepted	2	10
101	Great fit	700	2024-03-05	Pending	2	11

R6 — Contract

<u>contract_id</u>	start_date	end_date	status	proposal_id (FK, UNIQUE)
200	2024-02-10	2024-03-10	Active	100

R7 — Skill

<u>skill_id</u>	skill_name
1	HTML
2	CSS
3	JS

R8 — Freelancer_Skill

<u>freelancer_id</u> (FK)	<u>skill_id</u> (FK)	proficiency_level
2	1	Expert
2	2	Advanced
2	3	Expert

4.3.3 Why 2NF Is Satisfied

Each relation now has a single or composite primary key, and every non-key attribute depends on the **whole** primary key of its relation. No partial dependencies remain. The decomposition is lossless because each new relation preserves the functional dependency that motivated its creation.

4.4 Third Normal Form (3NF)

Rule. A relation is in 3NF if it is in 2NF **and** no non-key attribute is **transitively dependent** on the primary key through another non-key attribute.

4.4.1 Transitive Dependency Analysis

Table 4: Transitive dependency check for each 2NF relation

Relation	Analysis	TD Found?
User	All attributes (<code>name</code> , <code>email</code> , <code>password</code> , <code>registration_date</code> , <code>user_type</code>) depend directly on <code>user_id</code> . No non-key attribute determines another.	No
Client	Contains only the primary key <code>client_id</code> . No non-key attributes exist.	No
Freelancer	<code>bio</code> depends directly on <code>freelancer_id</code> . No transitive chain.	No
Project	All attributes depend directly on <code>project_id</code> . <code>client_id</code> is a foreign key but does not determine other non-key attributes within this relation.	No
Proposal	All attributes depend directly on <code>proposal_id</code> . The foreign keys <code>freelancer_id</code> and <code>project_id</code> do not create transitive chains among non-key attributes within this relation.	No
Contract	All attributes depend directly on <code>contract_id</code> . <code>proposal_id</code> is a foreign key with a UNIQUE constraint but does not cause transitive dependency.	No
Skill	<code>skill_name</code> depends directly on <code>skill_id</code> . No transitive chain.	No
Freelancer_Skill	<code>proficiency_level</code> depends directly on the composite key { <code>freelancer_id</code> , <code>skill_id</code> }. No transitive dependency.	No

4.4.2 3NF Result

The 2NF decomposition **already satisfies 3NF**. No transitive dependencies were found in any relation, and therefore no further decomposition is required.

This outcome is expected because the original system was designed with clean entity separation. The normalization process confirmed — rather than altered — the structure from 2NF onward, validating the quality of the initial database design.

4.5 Normalization Summary

The final schema is in **Third Normal Form (3NF)** because:

1. **1NF** — Every attribute in every relation holds only atomic values. No repeating groups exist. Each table has a well-defined primary key.
2. **2NF** — Every non-key attribute is fully functionally dependent on the entire primary key of its relation. No partial dependencies exist, even in relations with composite keys.
3. **3NF** — No non-key attribute transitively depends on the primary key through another non-key attribute. Every non-key attribute depends directly and only on the primary key.

The normalization process began from a single flat UNF table and, through systematic decomposition at each normal form stage, arrived at the same eight-relation schema that was independently derived through EER-to-relational mapping — validating both the design methodology and the normalization process.

5 Final Normalized Schema (3NF)

The following eight relations constitute the final normalized schema of the Freelance Marketplace System. Each relation is listed with its primary key (underlined), foreign keys (FK), and constraints explicitly marked.

1. **User** (user_id, name, email (UNIQUE), password, registration_date, user_type)
2. **Client** (client_id FK → User.user_id)
3. **Freelancer** (freelancer_id FK → User.user_id, bio)
4. **Project** (project_id, title, description, budget, date_posted, status, client_id FK → Client.client_id)
5. **Proposal** (proposal_id, cover_letter, bid_amount, date_submitted, status, freelancer_id FK → Freelancer.freelancer_id, project_id FK → Project.project_id)
6. **Contract** (contract_id, start_date, end_date, status, proposal_id FK → Proposal.proposal_id (UNIQUE))
7. **Skill** (skill_id, skill_name (UNIQUE))
8. **Freelancer_Skill** (freelancer_id FK → Freelancer.freelancer_id, skill_id FK → Skill.skill_id, proficiency_level)

5.1 Relationship Summary

Table 5: Relationships between database entities

Relationship	Type	Enforced By	Description
User — Client	1:1 (ISA)	PK = FK	Client inherits from User
User — Freelancer	1:1 (ISA)	PK = FK	Freelancer inherits from User
Client — Project	1:N	FK client_id	A client posts many projects
Freelancer — Proposal	1:N	FK freelancer_id	A freelancer submits many proposals
Project — Proposal	1:N	FK project_id	A project receives many proposals
Proposal — Contract	1:1	FK + UNIQUE	One contract per accepted proposal
Freelancer — Skill	M:N	Junction table	Via Freelancer_Skill with proficiency

5.2 Key Constraints

- **Primary Keys:** Every table has a primary key ensuring entity integrity. Freelancer_Skill uses a composite primary key (`freelancer_id`, `skill_id`).
- **Foreign Keys:** All relationships are enforced through foreign key constraints, maintaining referential integrity across the database.
- **UNIQUE Constraints:** `User.email` ensures no duplicate email addresses. `Skill.skill_name` prevents duplicate skill entries. `Contract.proposal_id` enforces the one-to-one relationship with Proposal.
- **NOT NULL Constraints:** Critical fields such as `User.name`, `Project.title`, `Proposal.bid_amount`, and `Contract.start_date` are required and cannot be left empty.

6 SQL Implementation

This section presents the complete SQL implementation of the Freelance Marketplace database, including database creation, table definitions with all constraints, sample data insertion, and essential queries demonstrating the database capabilities.

6.1 Database Creation

Creating the Database

```
CREATE DATABASE IF NOT EXISTS freelance_marketplace
  CHARACTER SET utf8
  COLLATE utf8_general_ci;

USE freelance_marketplace;
```

6.2 Table Definitions

6.2.1 User Table

The base entity for all system users. The `user_type` column distinguishes between Clients and Freelancers in the ISA hierarchy.

CREATE TABLE — User

```
CREATE TABLE 'User' (
  user_id          INT PRIMARY KEY AUTO_INCREMENT,
  name             VARCHAR(100) NOT NULL,
  email            VARCHAR(100) NOT NULL UNIQUE,
  password          VARCHAR(100) NOT NULL,
  registration_date DATE NOT NULL,
  user_type        VARCHAR(20) NOT NULL
);
```

6.2.2 Client Table

Implements the Client specialization. The primary key is also a foreign key referencing User, enforcing the 1:1 ISA relationship.

CREATE TABLE — Client

```
CREATE TABLE Client (
  client_id INT PRIMARY KEY,
  FOREIGN KEY (client_id)
    REFERENCES 'User' (user_id)
);
```

6.2.3 Freelancer Table

Implements the Freelancer specialization with an additional `bio` attribute. The same PK-as-FK pattern enforces the ISA relationship.

CREATE TABLE — Freelancer

```
CREATE TABLE Freelancer (  
    freelancer_id INT PRIMARY KEY,  
    bio          TEXT,  
    FOREIGN KEY (freelancer_id)  
        REFERENCES 'User'(user_id)  
);
```

6.2.4 Project Table

Stores projects posted by clients. The foreign key `client_id` establishes the 1:N relationship with Client.

CREATE TABLE — Project

```
CREATE TABLE Project (  
    project_id INT PRIMARY KEY AUTO_INCREMENT,  
    title      VARCHAR(150) NOT NULL,  
    description TEXT,  
    budget     DECIMAL(10,2),  
    date_posted DATE NOT NULL,  
    status     VARCHAR(50) NOT NULL,  
    client_id  INT NOT NULL,  
    FOREIGN KEY (client_id)  
        REFERENCES Client(client_id)  
);
```

6.2.5 Proposal Table

Records proposals submitted by freelancers for projects. Two foreign keys establish the relationships with Freelancer and Project.

CREATE TABLE — Proposal

```
CREATE TABLE Proposal (  
    proposal_id    INT PRIMARY KEY AUTO_INCREMENT,  
    cover_letter   TEXT,  
    bid_amount     DECIMAL(10,2) NOT NULL,  
    date_submitted DATE NOT NULL,  
    status         VARCHAR(50) NOT NULL,  
    freelancer_id  INT NOT NULL,  
    project_id     INT NOT NULL,  
    FOREIGN KEY (freelancer_id)  
        REFERENCES Freelancer(freelancer_id),  
    FOREIGN KEY (project_id)  
        REFERENCES Project(project_id)  
);
```

6.2.6 Contract Table

Represents formal agreements created from proposals. The UNIQUE constraint on `proposal_id` enforces the 1:1 relationship, ensuring each proposal can have at most one contract.

CREATE TABLE — Contract

```
CREATE TABLE Contract (  
    contract_id INT PRIMARY KEY AUTO_INCREMENT,  
    start_date  DATE NOT NULL,  
    end_date    DATE,  
    status      VARCHAR(50) NOT NULL,  
    proposal_id INT NOT NULL UNIQUE,  
    FOREIGN KEY (proposal_id)  
        REFERENCES Proposal(proposal_id)  
);
```

6.2.7 Skill Table

A lookup table for professional skills. The UNIQUE constraint on `skill_name` prevents duplicate entries.

CREATE TABLE — Skill

```
CREATE TABLE Skill (  
    skill_id    INT PRIMARY KEY AUTO_INCREMENT,  
    skill_name  VARCHAR(100) NOT NULL UNIQUE  
);
```

6.2.8 Freelancer_Skill Table (Junction Table)

Implements the M:N relationship between Freelancer and Skill. The composite primary key prevents duplicate assignments, and `proficiency_level` stores the relationship-specific data.

CREATE TABLE — Freelancer_Skill (Junction Table)

```
CREATE TABLE Freelancer_Skill (  
    freelancer_id      INT NOT NULL,  
    skill_id           INT NOT NULL,  
    proficiency_level  VARCHAR(50) NOT NULL,  
    PRIMARY KEY (freelancer_id, skill_id),  
    FOREIGN KEY (freelancer_id)  
        REFERENCES Freelancer(freelancer_id),  
    FOREIGN KEY (skill_id)  
        REFERENCES Skill(skill_id)  
);
```

6.3 Sample Data Insertion

6.3.1 Inserting Users

INSERT — User Records (6 users)

```
INSERT INTO 'User'
  (user_id, name, email, password,
   registration_date, user_type)
VALUES
  (1, 'Ahmed Hassan',
   'ahmed.client@example.com',
   'client123', '2026-03-01', 'Client'),
  (2, 'Sara Ali',
   'sara.client@example.com',
   'client123', '2026-03-04', 'Client'),
  (3, 'Omar Khaled',
   'omar.freelancer@example.com',
   'free123', '2026-03-05', 'Freelancer'),
  (4, 'Laila Samir',
   'laila.freelancer@example.com',
   'free123', '2026-03-06', 'Freelancer'),
  (5, 'Mona Adel',
   'mona.freelancer@example.com',
   'free123', '2026-03-08', 'Freelancer'),
  (6, 'Karim Nabil',
   'karim.client@example.com',
   'client123', '2026-03-10', 'Client');
```

6.3.2 Inserting Clients and Freelancers

INSERT — Client Records

```
INSERT INTO Client (client_id) VALUES
  (1), (2), (6);
```

INSERT — Freelancer Records (with bio)

```
INSERT INTO Freelancer (freelancer_id, bio) VALUES
  (3, 'Full-stack web developer with experience
       in PHP, MySQL, and responsive design.'),
  (4, 'Graphic designer focused on branding,
       UI layouts, and landing pages.'),
  (5, 'Content writer with experience in product
       descriptions and admin support.');
```

6.3.3 Inserting Projects

INSERT — Project Records (4 projects)

```
INSERT INTO Project
  (project_id, title, description, budget,
   date_posted, status, client_id)
VALUES
  (1, 'Business Website Redesign',
   'Need a clean redesign for a company website.',
   1200.00, '2026-04-01', 'Open', 1),
  (2, 'Logo and Brand Kit',
   'Design a logo and brand identity package.',
   450.00, '2026-04-03', 'In Review', 2),
  (3, 'Product Description Writing',
   'Engaging descriptions for 50 store items.',
   300.00, '2026-04-04', 'Open', 6),
  (4, 'Portfolio Website',
   'Personal portfolio for a photographer.',
   700.00, '2026-04-06', 'Closed', 1);
```

6.3.4 Inserting Proposals

INSERT — Proposal Records (5 proposals)

```
INSERT INTO Proposal
  (proposal_id, cover_letter, bid_amount,
   date_submitted, status,
   freelancer_id, project_id)
VALUES
  (1, 'I can redesign your business website.',
   1100.00, '2026-04-02', 'Accepted', 3, 1),
  (2, 'Strong experience in logo design.',
   420.00, '2026-04-04', 'Pending', 4, 2),
  (3, 'SEO-friendly product descriptions.',
   280.00, '2026-04-05', 'Pending', 5, 3),
  (4, 'Elegant photographer portfolio.',
   650.00, '2026-04-07', 'Rejected', 3, 4),
  (5, 'Modern portfolio with design details.',
   680.00, '2026-04-07', 'Pending', 4, 4);
```

6.3.5 Inserting Contracts

INSERT — Contract Record

```
INSERT INTO Contract
  (contract_id, start_date, end_date,
   status, proposal_id)
VALUES
  (1, '2026-04-08', '2026-04-25', 'Active', 1);
```

6.3.6 Inserting Skills and Assignments

INSERT — Skill Records (6 skills)

```
INSERT INTO Skill (skill_id, skill_name) VALUES
  (1, 'PHP'),
  (2, 'MySQL'),
  (3, 'HTML'),
  (4, 'CSS'),
  (5, 'Graphic Design'),
  (6, 'Content Writing');
```

INSERT — Freelancer_Skill Assignments (9 records)

```
INSERT INTO Freelancer_Skill
  (freelancer_id, skill_id, proficiency_level)
VALUES
  (3, 1, 'Advanced'),      -- Omar:  PHP
  (3, 2, 'Advanced'),      -- Omar:  MySQL
  (3, 3, 'Advanced'),      -- Omar:  HTML
  (3, 4, 'Intermediate'),  -- Omar:  CSS
  (4, 3, 'Intermediate'),  -- Laila: HTML
  (4, 4, 'Advanced'),      -- Laila: CSS
  (4, 5, 'Advanced'),      -- Laila: Graphic Design
  (5, 6, 'Advanced'),      -- Mona:  Content Writing
  (5, 3, 'Beginner');      -- Mona:  HTML
```

6.4 SELECT Queries with JOINS

6.4.1 List All Clients with User Information

SELECT — All Clients with Details

```
SELECT c.client_id,
       u.name,
       u.email,
       u.registration_date
FROM   Client c
INNER JOIN 'User' u
       ON c.client_id = u.user_id
ORDER BY c.client_id;
```

6.4.2 List All Freelancers with Bio

SELECT — All Freelancers with Bio

```
SELECT f.freelancer_id,
       u.name,
       u.email,
       u.registration_date,
       f.bio
FROM   Freelancer f
INNER JOIN 'User' u
       ON f.freelancer_id = u.user_id
ORDER BY f.freelancer_id;
```

6.4.3 List All Projects with Client Names

SELECT — Projects with Client Names (multi-table JOIN)

```
SELECT p.project_id,
       p.title,
       p.budget,
       p.status,
       p.date_posted,
       u.name AS client_name
FROM   Project p
INNER JOIN Client c
       ON p.client_id = c.client_id
INNER JOIN 'User' u
       ON c.client_id = u.user_id
ORDER BY p.project_id;
```

6.4.4 List All Proposals with Freelancer and Project Information

SELECT — Proposals with Related Entity Names

```
SELECT pr.proposal_id,
       pr.bid_amount,
       pr.status,
       pr.date_submitted,
       u.name AS freelancer_name,
       p.title AS project_title
FROM   Proposal pr
INNER JOIN Freelancer f
       ON pr.freelancer_id = f.freelancer_id
INNER JOIN 'User' u
       ON f.freelancer_id = u.user_id
INNER JOIN Project p
       ON pr.project_id = p.project_id
ORDER BY pr.proposal_id;
```

6.4.5 List All Contracts with Full Details

SELECT — Contracts with Freelancer and Project Details

```
SELECT ct.contract_id,
       ct.start_date,
       ct.end_date,
       ct.status,
       pr.bid_amount,
       u.name AS freelancer_name,
       p.title AS project_title
FROM   Contract ct
INNER JOIN Proposal pr
       ON ct.proposal_id = pr.proposal_id
INNER JOIN Freelancer f
       ON pr.freelancer_id = f.freelancer_id
INNER JOIN 'User' u
       ON f.freelancer_id = u.user_id
INNER JOIN Project p
       ON pr.project_id = p.project_id
ORDER BY ct.contract_id;
```

6.4.6 List All Freelancer Skills (M:N Traversal)

This query demonstrates the many-to-many relationship traversal through the junction table.

SELECT — Freelancer Skills via Junction Table

```
SELECT u.name AS freelancer_name ,
       s.skill_name ,
       fs.proficiency_level
FROM   Freelancer_Skill fs
INNER JOIN Freelancer f
       ON fs.freelancer_id = f.freelancer_id
INNER JOIN 'User' u
       ON f.freelancer_id = u.user_id
INNER JOIN Skill s
       ON fs.skill_id = s.skill_id
ORDER BY u.name , s.skill_name;
```

6.4.7 Count Proposals per Project (Aggregation)

SELECT — Proposal Count per Project (GROUP BY)

```
SELECT p.project_id ,
       p.title ,
       COUNT(pr.proposal_id) AS proposal_count
FROM   Project p
LEFT JOIN Proposal pr
       ON p.project_id = pr.project_id
GROUP BY p.project_id , p.title
ORDER BY proposal_count DESC;
```

6.4.8 Find Freelancers with a Specific Skill

SELECT — Freelancers with 'PHP' Skill (filtered JOIN)

```
SELECT u.name ,
       fs.proficiency_level
FROM   Freelancer_Skill fs
INNER JOIN Freelancer f
       ON fs.freelancer_id = f.freelancer_id
INNER JOIN 'User' u
       ON f.freelancer_id = u.user_id
INNER JOIN Skill s
       ON fs.skill_id = s.skill_id
WHERE  s.skill_name = 'PHP';
```

6.5 UPDATE Examples

6.5.1 Update a Project Status

UPDATE — Change Project Status to Closed

```
UPDATE Project
SET     status = 'Closed'
WHERE   project_id = 1;
```

6.5.2 Update a Proposal Status to Accepted

UPDATE — Accept a Proposal

```
UPDATE Proposal
SET     status = 'Accepted'
WHERE   proposal_id = 2;
```

6.5.3 Update a Freelancer's Bio

UPDATE — Modify Freelancer Bio

```
UPDATE Freelancer
SET     bio = 'Senior full-stack developer
              specializing in PHP, MySQL,
              and modern web frameworks.'
WHERE   freelancer_id = 3;
```

6.5.4 Update Contract End Date and Status

UPDATE — Complete a Contract

```
UPDATE Contract
SET     end_date = '2026-05-01',
        status   = 'Completed'
WHERE   contract_id = 1;
```

6.5.5 Update a Freelancer's Skill Proficiency

UPDATE — Change Proficiency Level

```
UPDATE Freelancer_Skill
SET     proficiency_level = 'Advanced'
WHERE   freelancer_id = 3
        AND skill_id = 4;
```

6.6 DELETE Examples

6.6.1 Delete a Freelancer-Skill Assignment

DELETE — Remove a Skill Assignment

```
DELETE FROM Freelancer_Skill
WHERE   freelancer_id = 5
      AND skill_id = 3;
```

6.6.2 Delete a Contract

DELETE — Remove a Contract

```
DELETE FROM Contract
WHERE   contract_id = 1;
```

6.6.3 Delete a Proposal (Respecting FK Order)

The related contract must be deleted first to satisfy the foreign key constraint.

DELETE — Remove a Proposal (cascade-aware)

```
-- Step 1: Delete the contract linked to this proposal
DELETE FROM Contract
WHERE   proposal_id = 1;

-- Step 2: Now safely delete the proposal
DELETE FROM Proposal
WHERE   proposal_id = 1;
```

6.6.4 Delete a Skill (With Dependent Records)

All freelancer-skill assignments for this skill must be removed first.

DELETE — Remove a Skill (with assignments cleanup)

```
-- Step 1: Remove all assignments for this skill
DELETE FROM Freelancer_Skill
WHERE   skill_id = 6;

-- Step 2: Now safely delete the skill itself
DELETE FROM Skill
WHERE   skill_id = 6;
```

7 Conclusion

The Freelance Marketplace System database project successfully demonstrates the application of fundamental database design principles and relational database theory in modeling a real-world freelancing platform.

7.1 Database Design

The conceptual design phase produced a clear Entity Relationship Diagram (ERD) and an Enhanced Entity Relationship Diagram (EERD) that accurately captured the system's entities, attributes, and relationships. The EERD specifically modeled the ISA specialization hierarchy for User, Client, and Freelancer, demonstrating the use of generalization and specialization in database design.

7.2 Relational Schema

The EER-to-relational mapping process translated the conceptual model into an eight-table relational schema. Key design decisions include:

- Using the primary-key-as-foreign-key pattern for the ISA subtypes (Client and Freelancer), ensuring a clean one-to-one relationship with the User base table.
- Implementing the many-to-many relationship between Freelancer and Skill through the Freelancer_Skill junction table with a composite primary key and an additional proficiency_level attribute.
- Enforcing the one-to-one relationship between Proposal and Contract through a UNIQUE constraint on the foreign key.

7.3 Normalization

The systematic normalization process, progressing from UNF through 1NF, 2NF, and 3NF, confirmed that the final schema is free from repeating groups, partial dependencies, and transitive dependencies. The fact that the normalization independently produced the same eight-table structure as the EER mapping validates both approaches and demonstrates the consistency and correctness of the design.

7.4 SQL Implementation

The SQL implementation faithfully translates the relational schema into executable DDL statements with appropriate constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL). The sample data insertion demonstrates the proper order of operations respecting referential integrity. The SELECT queries with multiple JOINS showcase the database's ability to efficiently retrieve related information across tables, while the UPDATE and DELETE examples demonstrate proper data modification with foreign key awareness.

7.5 Data Integrity

Throughout the design and implementation, data integrity is maintained at multiple levels:

- **Entity integrity** through primary keys on all tables.
- **Referential integrity** through foreign key constraints linking related tables.
- **Domain integrity** through appropriate data types and NOT NULL constraints.
- **Semantic integrity** through the UNIQUE constraint on Contract.proposal_id, User.email, and Skill.skill_name.

This project demonstrates that careful application of database design methodology — from conceptual modeling through normalization to physical implementation — produces a robust, consistent, and maintainable database system suitable for managing the operations of a freelance marketplace platform.

Project Website

The implemented system can be accessed online through the following link:

<https://freelance-marketplace.42web.io//>